Why RAG gets it wrong

10 checks for a more reliable pipeline

A practical checklist for your next evaluation.

A wrong answer has more than one possible cause

Retrieval-augmented generation (RAG) gives an LLM retrieved information to help answer a question.

The evidence may be missing or incomplete. The response may still go beyond it.

Inspect the retrieved text as well as the response.

01

Evaluation

A useful test set makes failures visible

Keep a set of real questions and the evidence each answer needs. Score retrieval and answer quality separately.

WHAT TO CHECK

Did we find the evidence?
Did the answer use it correctly?

Review a sample yourself to check the model judge.

02

Chunking

A passage needs enough context to stand alone

Use document structure to guide the split. Keep a rule near its exceptions and preserve the heading that explains it.

EXAMPLE

A refund rule loses meaning
when its exception lands
in another chunk.

Compare chunk sizes on the same question set.

03

Hybrid search

Exact words and semantic meaning both matter

Combine keyword retrieval, such as BM25, with vector retrieval. Merge the results before choosing the context.

EXAMPLE

“ZX-71 warranty coverage”

The product code needs an exact match. The wording around coverage may vary.

04

Query rewriting

A user question may need a clearer search query

Use the conversation context to turn a vague question into a clearer search query. Multiple query variants can explore different wording while keeping the original intent.

EXAMPLE

“Why can't I get in?”
becomes
“Account sign-in troubleshooting”

Check that the rewrite keeps the user's intended subject.

05

Reranking

Retrieved candidates still need a relevance check

A cross-encoder reads the query and each candidate together. Use its scores to reorder a shortlist before generation.

EXAMPLE

A general account guide
can rank below the page
about the exact login error.

Measure the relevance gain alongside added latency.

06

HNSW settings

Approximate search balances speed and recall

M controls graph connectivity. ef_construction affects index building. Search ef controls how broadly the query explores the graph.

WHAT TO COMPARE

Retrieved evidence
versus
query latency and memory

Benchmark settings on your own vectors and queries.

07

Embedding fit

Similarity depends on the task and vocabulary

Test an embedding model on your domain queries first. If it misses relevant passages, compare alternatives or fine-tune with suitable examples.

EXAMPLE

“Restart the service”
and
“Reinitialize the worker”

Use held-out queries to check whether training helps.

08

Conversation memory

A follow-up question often leaves context unstated

Keep relevant conversation context available. Use persistent memory when useful information must carry across sessions.

EXAMPLE

“Does that apply to me?”
needs the earlier plan
or product being discussed.

Trim stale context so it does not distract from the current question.

09

Semantic caching

Similar questions can reuse a suitable answer

Compare an incoming question with cached questions. Reuse a response only when the match and its context are appropriate.

EXAMPLE

“Where is my invoice?”
and
“How can I find my invoice?”

Set an expiry and keep user-specific responses scoped correctly.

10

LLM fine-tuning

Response behavior deserves its own evaluation

If the evidence is useful but the response format still fails, compare prompting with task-specific fine-tuning. LoRA trains a small set of adapter parameters.

EXAMPLE

Consistent output structure for a clearly defined task

Test source support separately. Training does not guarantee a grounded answer.

A quick diagnostic

Missing exact identifiers
Hybrid search

Incomplete passages
Chunking

Relevant evidence ranked too low
Reranking

Weak results on domain queries
Embedding evaluation

Lost context in follow-up questions
Conversation memory

The next step is a failed question

Keep the question, retrieved passages and answer together. Choose one change to investigate, then rerun the same test set.

Which failure would you investigate first?

Further reading

Qdrant
Hybrid retrieval and HNSW search
qdrant.tech/documentation

Sentence Transformers
Retrieval, reranking and embedding training
sbert.net

LangChain
Chunking and conversation memory
docs.langchain.com

Ragas
RAG evaluation metrics
docs.ragas.io

Redis
Semantic caching
redis.io/docs

Hugging Face PEFT
LoRA and parameter-efficient adaptation
github.com/huggingface/peft

Ma et al., EMNLP 2023
Query rewriting for retrieval
arxiv.org/abs/2305.14283